Software Design Document

Travel From Photo

Team Members

Younghak Yoo

(Younghak.Yoo@stonybrook.edu)

Jaemin Jeon

(Jaemin.Jeon@stonybrook.edu)

Course: CSE 416

Instructor: Professor Yoon Seok Yang

Date: June 1, 2026

Revision Summary

This revised version reflects the updated development direction. The project is now focused on the features that the team plans to keep: Search, Gallery, Journal, Live Chat, Login, Settings, Security, Database, and Deployment. Travelize and other extra features were removed from the main scope.

While we initially considered simplifying the search feature using a single service, we ultimately implemented a Hierarchical Scoring Pipeline to deliver highly accurate locations and minimize user friction. By strategically distributing multiple APIs across distinct tiers and differentiating their confidence score weights, the system aggregates and extracts the most reliable location candidates. Filtered recommendations are then presented to the user. If the correct location is not found among the suggestions, users can trigger a re-search or manually enter the coordinates, ensuring a foolproof and seamless user experience.

1. Introduction

1.1 Product Description

Travel photos preserve valuable memories, but the location information associated with those photos is often lost over time. When images are uploaded to social media, transferred between devices, or stored on cloud services, metadata such as GPS coordinates may be removed. As a result, users frequently remember the moment captured in a photo but struggle to recall where it was taken. Recovering this information manually can require significant time and effort.

Travel From Photo is an AI-powered web application designed to help users rediscover the locations behind their travel memories. By analyzing the visual content of uploaded images, the system identifies possible locations and presents users with reliable location suggestions. Rather than requiring users to search through maps, travel records, or photo archives, the system simplifies the process by transforming a single photo into actionable location information.

To improve accuracy and reduce user frustration, the system utilizes a multi-tier location inference pipeline. Multiple image analysis and location recognition services contribute evidence that is evaluated and combined to produce highly confident location candidates. This approach allows the system to provide more reliable results than relying on a single source of information.

Beyond location recovery, the platform helps users preserve and organize their travel experiences. Photos, recovered locations, and generated travel content can be stored in a unified environment, allowing users to revisit and manage their memories more effectively. By combining artificial intelligence, image understanding, and travel memory management, Travel From Photo provides a practical solution for reconnecting users with places they may have forgotten.

Main features of our system:

1. **Search**
- Upload maximum five pictures and extracts GPS data.
- Collect evidence of location through four google vision api(Logo, Web, OCR, Landmark)
- If there are no common locatio candidates, extract location clues from OpenAI.
- Synthesize all clues to derive the final location candidates
- The user selects the desired location
- If the desired location is not among the candidates, the user can request a re-search

2. **Gallery**
- Save searched photos and user-selected locations.
- Organize photos by upload session, date, or selected location.
- Allow users to view, rename, and delete saved image groups.

3. **Journal**
- Use saved photos and selected location data to generate a simple trip journal.
- The system uses the CLIP API to analyze uploaded photos and extract the most relevant tags.
- Next, OpenAI generates a descriptive journal entry based on the tags and image analysis.
- The user is provided with a complete journal that combines all generated information.
- Users can save their journals and access them at any time.
- The extracted tags are also used to generate visual statistics, allowing users to understand their
  travel experiences and patterns at a glance.

4. **Live Chat(**수정 필요**)**
- Provide an in-app chat area for user support or user communication.
- Store chat messages only for authenticated users.

5. **Login and Settings**
- Allow users to create an account and sign in.
- Allow users to manage account settings and simple service preferences.

6. **Security, Database, and Deployment**
- Protect uploaded images and user data.
- Store users, images, selected locations, journals, and chat records in the database.
- Deploy the frontend, backend, and database in a stable server environment

1.2 Scope

The system is a web-based application that operates through standard web browsers and does not require users to install any mobile or desktop software. The application is designed to support desktop, laptop, tablet, and mobile devices.

The primary scope of the project is to provide an AI-powered photo location recovery service. Users can upload up to five travel photos at a time, and the system analyzes visual information using a multi-tier location inference pipeline. The pipeline combines metadata extraction, multiple Google Vision services, and OpenAI-based image analysis to identify possible locations. The system presents ranked location candidates to the user, who can then select the most appropriate result. If none of the suggested locations are correct, users may initiate a re-search process or manually provide the location information.

The project also includes a gallery management component. Uploaded photos and verified locations are stored and organized for future access. Users can browse their saved records, manage image collections, and maintain a structured archive of travel memories.

A journal generation feature is included within the project scope. The system analyzes saved travel photos using the CLIP API to extract meaningful tags related to subjects, moods, and activities. These tags, together with image analysis results, are used by OpenAI to generate editable travel journal content. Generated journals can be saved and reviewed later. The extracted tags are additionally used to create visual summaries and statistics that help users understand overall travel patterns and experiences.

The system provides a live chat feature that supports communication within the platform. Chat functionality is available only to authenticated users, and messages are stored securely as part of the user account.

User authentication and account management are also included in the project scope. Users can create accounts, sign in securely, manage personal settings, and customize basic service preferences. Appropriate security measures are implemented to protect uploaded images, personal information, and application data.

The project includes backend database integration for storing user accounts, uploaded images, selected locations, generated journals, and chat records. Finally, the complete system, including the frontend, backend, and database services, is deployed in a stable cloud environment to ensure accessibility and reliability.

The current scope does not include automatic itinerary generation, route planning, hotel recommendations, transportation booking, or other advanced travel planning services. These features are considered outside the boundaries of the present project and may be explored in future development.

1.3 Users

The first group of users is travelers. These users take many photos during trips and may lose location data later. They use the system to find possible locations and save the correct result in their gallery.

The second group is students and general users. These users may find images online or have old personal photos. They use the system mainly for location search and simple organization

The third group is users who want to make a simple memory record. These users use the journal feature after saving photos and locations in the gallery.All users are expected to have basic computer skills.

They should be able to use a browser, upload an image, choose a result, and edit text fields.

1.4 User Feedback

User feedback was collected through informal interviews and surveys with students and travelers in our network. Several users reported that they could not remember where a specific photo was taken after returning from a trip. When asked about existing tools, most had tried Google Lens at least once. The common complaint was that results often returned broad region names or unrelated landmarks, leaving users unsure which result was correct.

Users also said they wanted a focused result with a clear confidence level, not a long list of possibilities. A few users mentioned that being able to manually correct a wrong AI result would make them trust the tool more.

These responses shaped two core decisions: keeping the candidate list short and always providing a manual input fallback.

1.5 Existing Alternatives

1. **Google Lens** Google Lens can identify objects, text, and landmarks from an image. It is widely used and handles a broad range of image types. However, it is designed as a general-purpose visual search tool, not a travel photo organizer. Results are often broad and lack confidence scores, so the user must judge accuracy on their own. There is also no way to save a selected result or connect it to a personal photo library.

2. **GeoSpy** GeoSpy estimates geographic location from visual clues in an image. It is closer to our use case, but result accuracy drops significantly when images lack distinctive landmarks. Certain features also require a paid plan, which limits accessibility for casual users.

3. **Our Improvement** Travel From Photo differs from these tools in two ways. First, it is built specifically for the travel photo workflow: upload, review candidates with confidence scores, select or manually correct, then save to a personal gallery. Second, every search result is stored and linked to the user's account, making it useful not just for finding a location but for building an organized travel record over time. Neither Google Lens nor GeoSpy offers this connected save-and-organize flow.

1.6 Definitions

Travel Photo: An image uploaded by the user that was taken during a trip. The system uses this image as the input for location search. Supported formats are JPEG, PNG, and WEBP.

Image Analysis: The process of extracting visual clues from a photo using the OpenAI API. The system reads landmarks, signs, objects, and other visible features to generate candidate locations.

Candidate Location: A possible place identified by the image analysis process. Each candidate includes a place name, general region, confidence score, and a short explanation of why it was suggested. The system may return multiple candidates for one image.

Confidence Score: A numerical value that indicates how strongly the system believes a candidate location matches the uploaded photo. Higher scores mean stronger evidence. This score is stored with the image and displayed to the user during result review.

Source Type: A label that records how the final location was determined. The value is either AI-suggested, meaning the user selected one of the system's candidates, or manually entered, meaning the user typed the location themselves. This value is stored with every saved image.

Selected Location: The final location attached to an image after the user either chooses a candidate or enters one manually. This is the location that appears in the gallery and is used as input for journal generation.

Gallery Group: A collection of images saved by the user, organized by upload session or location. Each group has a title, creation date, and a list of associated images with their selected locations. Users can rename or delete groups.

Journal: An AI-generated travel record created from saved gallery images and their selected locations. The system uses the OpenAI API to produce a draft, which the user can edit before

saving. Each journal has a title, content, visibility setting, and a reference to the images it was built from.

Visibility: A setting that controls who can see a journal or gallery group. The default value is private, meaning only the owner can access it.

Manual Location Input: A fallback option available when none of the AI-suggested candidates are correct. The user can type a country, city, place name, or address. The system saves this input as the selected location with source type marked as manually entered.

Authentication Token: A secure credential issued by the backend after a successful login. The frontend includes this token in subsequent requests to access protected features such as search, gallery, and journal.

File Validation: A server-side check performed on every uploaded image before analysis begins. The system verifies file format, file size, and basic integrity. Files that fail validation are rejected before reaching the OpenAI API.

Supported Format: Image file types accepted by the system: JPEG, PNG, and WEBP. Other formats are rejected at upload.

Image Size Limit: The maximum file size the system will accept for a single upload. Files exceeding this limit are rejected to maintain server stability.

1.7 References

OWASP Foundation. File Upload Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

OWASP Foundation. Input Validation Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

OpenAI. OpenAI API Documentation. https://platform.openai.com/docs

OpenAI. GPT-4o Vision Capabilities. https://platform.openai.com/docs/guides/vision

FastAPI. FastAPI Documentation. https://fastapi.tiangolo.com

SQLAlchemy. SQLAlchemy Documentation. https://docs.sqlalchemy.org

Alembic. Alembic Documentation. https://alembic.sqlalchemy.org

PostgreSQL. PostgreSQL Documentation. https://www.postgresql.org/docs

React. React Documentation. https://react.dev

Docker. Docker Documentation. https://docs.docker.com

Google Developers. Google Maps JavaScript API. https://developers.google.com/maps/documentation/javascript

Leaflet. Leaflet Documentation. https://leafletjs.com

2. Requirements

2.1 Functional Requirements

1. Search

User Interaction Requirements

- The user shall be able to access the Search feature from the main navigation interface after login.
- The user shall be able to upload a single travel image from their device.
- The system shall accept only JPEG, PNG, and WEBP image formats.
- The system shall reject files that exceed the maximum allowed size of 30MB.
- The system shall reject files that are corrupted or cannot be decoded.
- The user shall be able to provide an optional country or city hint before starting the search.
- The user shall be able to start the search after a valid image has been uploaded.
- The system shall display a loading animation while the backend processes the image.

Image Validation Requirements

- The system shall verify that the uploaded file is a supported image format before processing.
- The system shall verify that the file size does not exceed the allowed limit.
- The system shall verify that the image is not corrupted and can be successfully decoded.
- The system shall reject invalid files and notify the user with a specific error message before any analysis begins.

Image Analysis Requirements

- The system shall check for EXIF GPS metadata immediately after validation. If GPS coordinates are present, the system shall use them as the primary location source with a confidence score of 1.0 and skip further analysis.

- If GPS data is absent, the system shall prepare two versions of the image: the original for web detection and AI analysis, and a preprocessed copy with brightness, blur, and rotation corrections applied.
- The system shall run the following detectors in parallel using the preprocessed image: OCR, Logo Detection, Web Detection, and Landmark Detection.
- The system shall assign base confidence weights to each signal source: GPS at 1.0, Landmark Detection and AI at 0.75, Web Detection and Logo Detection at 0.70, and OCR at 0.50.
- The system shall discard low-quality clues including URL fragments, strings that are too short, and strings that are excessively long before candidate generation.
- The system shall attempt to resolve remaining clues into real places using the Places API. Clues that cannot be resolved to a real place shall be discarded.
- The system shall merge candidates that refer to the same location. Candidates sharing the same Google Place ID shall be merged first. If no Place ID match exists, candidates within 1km of each other shall be grouped. If neither condition applies, candidates with the same normalized name shall be merged.
- The system shall absorb broader location candidates into more specific ones when they refer to the same place at different levels of detail.
- The system shall calculate a score for each candidate by summing the product of confidence weight and certainty for each contributing signal. A diversity bonus shall be applied when signals from different detection mechanisms agree on the same candidate.
- The system shall apply user-provided hints to adjust candidate scores. Candidates matching the hinted country shall receive a score multiplier of 1.3. Candidates matching the hinted city shall receive a multiplier of 1.2. Candidates that contradict the hinted country shall receive a multiplier of 0.4, and those contradicting the hinted city shall receive a multiplier of 0.5.
- If Web Detection returns strong evidence that the exact image exists online, the system shall immediately confirm that candidate as the result without calling the AI model.
- If no candidate meets the confidence threshold after the parallel analysis stage, the system shall call the OpenAI API with the original image. Any new candidates returned by the AI shall be merged into the existing candidate pool and re-scored through the same scoring pipeline.
- If confidence remains insufficient after the first AI call, the system shall make a second AI call with all accumulated clues to produce a final ranked result.

- If multiple images are submitted together, the system shall perform cross-image analysis after all individual analyses are complete. Coordinates from images with a score above 0.5 shall be used to calculate a geographic center, which shall be applied as a location prior for lower-confidence images in the same group.
- If low-confidence images remain after cross-image analysis, the system shall call the AI again with the confirmed locations from other images in the group as additional context.

Result Presentation Requirements

- The system shall classify the final result into one of four verdict levels: Confident when the top candidate scores sufficiently high with a clear gap from the second candidate, Likely when the top candidate scores adequately but the gap is narrow, Suggestions when only weak candidates exist, and Failed when no candidates could be generated.
- The system shall display the verdict level clearly alongside the result.
- Each candidate shall include a place name, general region, confidence score, and short reason.
- Candidates shall be displayed in descending order by score.
- The system shall clearly state that results are estimated suggestions, not guaranteed answers.
- The user shall be able to select one candidate as the final location.
- The user shall be able to reject all candidates if none are correct.
- The user shall be able to correct the result by editing the place name directly.
- The user shall be able to reposition the location by selecting a point on a map.

Manual Location Input Requirements

- The system shall provide a manual location input field when no candidate is correct or when the verdict is Failed.
- The user shall be able to enter a country, city, place name, or address manually.
- The system shall validate that the manual input is not empty before saving.
- The system shall save the manually entered location as the final selected location.
- The system shall mark whether the final location was AI-suggested or manually entered using the source type field.

Gallery Integration Requirements

- The system shall save the uploaded image to the user's gallery after the search process completes.
- The system shall store the selected or manually entered location with the image.
- The system shall store the confidence score, verdict level, and source type alongside the saved image.
- The system shall allow the saved image to appear in the user's gallery grouped by session or location.
- The system shall not save an image without a final selected or manually entered location.

Exception Handling Requirements

- The system shall notify the user if the upload fails due to a network or server error.
- The system shall notify the user if the file is rejected due to format, size, or corruption.
- The system shall notify the user if the OpenAI API request fails.
- The system shall allow the user to retry the search after any failure.
- The system shall allow manual location input when AI candidates are unavailable or the API call fails.

2. Gallery Management

Gallery Requirements

- The user shall be able to access the Gallery feature after login.
- The system shall display saved image groups as a city-based card grid in descending order by creation date.
- Each gallery group card shall display a thumbnail, city name, country, and image count.
- The user shall be able to select a group to view its detail page.
- The group detail page shall display a photo grid with a click-to-open image viewer supporting left and right navigation.
- The group detail page shall display the city, country, and image count at the top.
- The user shall be able to view a map showing the GPS positions of images that have location coordinates, rendered using Leaflet.
- The user shall be able to update the title of a gallery group.
- The user shall be able to delete a gallery group and all images within it.
- The system shall show saved location information for each image when available.

- The user shall be able to request re-analysis of an image's location using the Verify Location button, which re-runs the search pipeline on that image.
- The user shall be able to set a privacy level per gallery group. A group may be set to public or private independently of other groups.

Storage Requirements

- The system shall store each gallery group with a unique identifier, owner reference, title, creation date, and update date.
- The system shall store each image with a reference to its gallery group and owner.
- The system shall store the upload date, file path, selected location, confidence score, verdict level, and source type for each image.
- The system shall keep each user's gallery private by default.

3. Generate AI Trip Journal

Journal Generation Requirements

- The user shall be able to access the Journal feature after login.
- The user shall be able to select images from their gallery to use as journal source material.
- The system shall only allow images that contain valid metadata to be used for journal generation. Images without metadata shall be skipped with a notification to the user.
- The system shall use the Places API to convert GPS coordinates from selected images into human-readable addresses.
- The system shall use the CLIP API to analyze each selected image and extract descriptive tags.
- The system shall check whether selected images share the same date before calling external APIs to avoid redundant requests.
- The system shall use GPT-4o mini to convert extracted tags and location data into natural language journal content.
- Each journal entry shall include the photo, place information, keywords, date, time, and estimated travel time and transportation method between consecutive photos.
- The generated journal shall be shown as a draft before saving.

Statistics and Recommendation Requirements

- The system shall generate travel statistics from the user's journal and photo data, including visited countries, tag distribution, and time patterns.
- The system shall visualize statistics using a minimum of three charts: a summary stats card, a world map of visited locations, and a tag distribution pie chart.
- The system shall pass extended statistics data to the OpenAI API to generate personalized next destination recommendations. This data shall include top cities, average trip duration, most frequent travel months, continental distribution, average latitude of visited locations, and pattern changes over the last six months.
- Each recommendation shall include a destination name and an explicit reason tied to the user's travel statistics.
- The system shall only generate recommendations when the user has three or more journal collections. If fewer collections exist, the system shall notify the user that more data is needed and provide a partial result with an explanation of its limitations.

Editing and Storage Requirements

- The user shall be able to edit the generated journal text before saving.
- The user shall be able to edit the title and notes.
- The system shall store journals with a unique identifier, author ID, title, content, visibility, creation date, and update date.
- The default visibility of a journal shall be private.
- The user shall be able to update or delete saved journals.
- The system shall store a reference to the source images used to generate each journal.

4. Live Chat

Chat Requirements

- The user shall be able to access Live Chat after login.
- The user shall be able to send and receive text messages.
- The system shall show message time and sender information.
- The system shall store chat messages in the database.
- The system shall restrict chat access to authenticated users.

5. Login and Account

Authentication Requirements

- The user shall be able to create an account.
- The user shall be able to log in and log out.
- The system shall protect routes that require authentication.
- The system shall store passwords securely using hashing.
- The system shall use tokens or secure session handling for authenticated requests.

6. Settings

Settings Requirements

- The user shall be able to access a settings page after login.
- The user shall be able to update basic profile information.
- The user shall be able to change simple preferences such as display name or default privacy.
- The system shall validate all settings input before saving.

7. Security

Security Requirements

- The system shall validate uploaded files on the client and server.
- The system shall reject unsupported files and oversized files.
- The system shall restrict private images, galleries, journals, and chats to their owners.
- The system shall protect API keys through environment variables.
- The system shall not expose OpenAI API keys to the frontend.
- The system shall sanitize user input to reduce injection and scripting risks.

8. Database

Database Requirements

- The system shall store users, images, gallery groups, search results, journals, settings, and chat messages.

- The database shall support relationships between users and their saved content.
- The system shall use migration scripts or schema setup scripts for deployment.
- The system shall support backup or export procedures when possible.

9. Deployment

Deployment Requirements

- The frontend shall be deployable as a web application.
- The backend shall run as an API server.
- The database shall run in a server environment.
- Environment variables shall be used for API keys and database settings.
- The system should use Docker or deployment scripts to make setup consistent.

2.2 Use cases

Use Case 1: Image-Based Location Search Actor: User Priority: Essential

Primary Scenario

1. The user opens the Search page from the main navigation.
2. The user uploads a single travel image from their device.
3. The system validates the file format, size, and integrity.
4. The system displays a loading animation while the backend begins processing.
5. The backend checks for EXIF GPS metadata. If GPS coordinates are found, the system immediately resolves the location and proceeds to step 12.
6. If GPS is absent, the backend prepares two image versions: the original and a preprocessed copy with brightness, blur, and rotation corrections applied.
7. The backend runs OCR, Logo Detection, Web Detection, and Landmark Detection in parallel.
8. The system filters out low-quality clues and resolves remaining signals into real place candidates using the Places API.
9. The system merges duplicate candidates, absorbs broader locations into more specific ones, and calculates a score for each candidate based on signal source weights and diversity bonuses.

10. The system applies any user-provided country or city hint to adjust candidate scores.
11. If Web Detection returns strong evidence that the exact image exists online, the system confirms that candidate immediately without calling the AI model. Otherwise, the system calls the OpenAI API with the original image and re-scores all candidates.
12. The system classifies the result as Confident, Likely, Suggestions, or Failed based on the top candidate score and gap to the second candidate.
13. The system displays the ranked candidate list with place name, region, confidence score, verdict level, and short reason for each candidate.
14. The user selects one candidate as the final location, or edits the place name directly, or repositions the pin on the map.
15. The system saves the image with the selected location, confidence score, verdict level, and source type to the user's gallery.

Extension Scenarios

- Invalid file format, size, or corruption: the system rejects the file before processing and displays a specific error message.
- EXIF GPS found: the system skips the parallel analysis pipeline and immediately confirms the location with a confidence score of 1.0.
- Web Detection confirms exact image match: the system skips the AI call and confirms the result immediately.
- OpenAI API failure: the system notifies the user, allows retry, and provides the manual input option.
- No correct candidate found: the user enters a country, city, place name, or address manually. The system saves this as the final location with source type marked as manually entered.
- Multiple images submitted: after individual analysis, the system performs cross-image analysis using confirmed coordinates to assist lower-confidence images in the same group.

Use Case 2: Manage Gallery Actor: User Priority: Essential

Primary Scenario

1. The user opens the Gallery page after login.
2. The system displays saved image groups as a city-based card grid in descending order by creation date, showing thumbnail, city, country, and image count for each group.
3. The user selects a group to open its detail page.
4. The system displays a photo grid with saved location information for each image.
5. The user clicks an image to open the image viewer with left and right navigation.
6. The user clicks the map button to view GPS positions of images on a Leaflet map.
7. The user renames the group title if needed.
8. The user clicks the Verify Location button on a specific image to trigger re-analysis using the full search pipeline.
9. The system updates the image's location, confidence score, and verdict level with the new result.
10. The user sets the privacy level of the group to public or private.
11. The user deletes the group if needed. The system removes the group and all associated images.

Extension Scenarios

- No gallery groups exist: the system displays an empty state message with a prompt to start a search.
- Invalid or empty title: the system prevents the update and displays an error message.
- Verify Location returns a different result: the system updates the stored location and notifies the user of the change.
- Verify Location fails: the system notifies the user and retains the existing location data.
- Group is set to private: the system restricts access to the owner only.

Use Case 3: Generate AI Trip Journal Actor: User Priority: Expected

Primary Scenario

1. The user opens the Journal page after login.
2. The user starts a new journal and selects saved images from their gallery.
3. The system checks each selected image for valid metadata. Images without metadata are skipped and the user is notified.

4. The system uses the Places API to convert GPS coordinates from selected images into human-readable addresses.
5. The system uses the CLIP API to extract descriptive tags from each selected image.
6. The system uses GPT-4o mini to generate natural language journal content from the extracted tags, location data, and any optional notes provided by the user.
7. The generated draft is displayed to the user, including photo, place information, keywords, date, time, and estimated travel time and transportation method between consecutive photos.
8. The user edits the draft text, title, and notes as needed.
9. The user saves the journal. The system stores it with private visibility by default.
10. The user navigates to the statistics view, which displays a summary stats card, a world map of visited locations, and a tag distribution pie chart.
11. The system passes extended statistics to the OpenAI API and displays personalized next destination recommendations with explicit reasons tied to the user's travel patterns.

Extension Scenarios

- Selected images have no metadata: the system skips those images, notifies the user, and continues with valid images.
- Fewer than three journal collections exist: the system notifies the user that recommendations may be limited and provides a partial result with an explanation.
- Journal generation fails: the system notifies the user and allows retry.
- User edits and saves the journal: the system overwrites the stored content with the updated version.
- User deletes a journal: the system removes the journal and its reference to source images.

Use Case 4: Live Chat Actor: User Priority: Expected

Primary Scenario

1. The user opens the Live Chat page after login.
2. The system loads and displays existing chat messages in chronological order, showing sender information and timestamp for each message.
3. The user types and sends a message.

4. The system stores the message in the database and displays it in the chat view immediately.
5. Other authenticated users in the chat receive the message in real time via WebSocket.

Extension Scenarios

- Network error during send: the system displays a failed-send indicator on the message and allows the user to retry.
- User is not authenticated: the system redirects to the login page.
- Message exceeds length limit: the system prevents submission and notifies the user.

Use Case 5: Login and Settings Actor: User Priority: Essential

Primary Scenario

1. The user accesses the login page. If the user does not have an account, they navigate to the registration page.
2. For registration, the user enters required profile information. The system validates the input and creates the account.
3. The user logs in with their credentials. The system validates the credentials and issues an authentication token.
4. The frontend stores the token and uses it for all subsequent protected requests.
5. The user opens the Settings page.
6. The user updates their display name, email, or default privacy preference.
7. The system validates the input and saves the changes.
8. The user logs out. The system invalidates the token and redirects to the login page.

Extension Scenarios

- Wrong credentials: the system displays an authentication error and does not issue a token.
- Registration with an already existing email: the system displays a conflict error and prompts the user to log in instead.
- Invalid settings input: the system highlights the invalid field and prevents saving until corrected.
- Expired authentication token: the system redirects the user to the login page.

2.4 Non-Functional Requirements

Operating constraints

- The system requires an active internet connection to function.
- The system depends on the OpenAI API for image location reasoning and journal text generation, and on the Google Maps Platform (Vision, Places, Geocoding APIs) for landmark detection and location resolution. If either service is unavailable, the affected feature degrades gracefully while the rest of the system remains operational.
- The backend (FastAPI) and a PostgreSQL database must be running for the system to function.
- Users must access the system through a modern web browser that supports current web standards (ES2020+, Fetch API, Geolocation).
- Image upload is accepted only for supported raster formats (JPEG, PNG, WebP). The server rejects files exceeding 25 MB before processing begins.
- Login is required to use the Search, Gallery, Journal, and Chat features.
- Estimated travel routes and journal descriptions may be generated without live traffic or real-time transportation data.

Platform constraints

- The system is implemented as a web application requiring no client-side installation.
- The system runs in modern desktop and mobile web browsers without modification.
- The frontend, backend, and database run as separate services within a consistent container environment, managed by Docker Compose.
- The frontend communicates with the backend through a single HTTP entry point (/api), proxied by Nginx. The backend and database are not exposed directly to clients in production.

Accuracy and Precision

- The system infers possible locations from an uploaded image using available clues including EXIF GPS coordinates, landmark detection, logo detection, OCR text, web entity matching, and visual similarity (CLIP).
- The system presents multiple ranked candidate locations rather than forcing a single exact result.

- The system allows approximate or similar place suggestions when exact identification is not possible.
- The system does not require geographic coordinates to produce a result.
- Results should be more accurate when an image contains identifiable landmarks, embedded GPS metadata, or recognizable text or logos.
-

Portability

- The system runs on common modern browsers without code changes.
- The system is deployable to a different server environment through configuration only (environment variables), without modifying source code.
- Core components run unchanged when the system is moved to another environment.
- External API credentials are switchable through environment variables.
- The database connection is configurable per environment through a single DATABASE_URL variable.

Reliability

- The system shall remain operational across at least 95% of user sessions.
- A failed image upload or analysis shall not interrupt other system functions.
- The system shall display a clear error message when an upload or analysis operation fails.
- When a single external signal (one API call) fails during location analysis, the system falls back to remaining signals and still returns a best-effort result. Each pipeline tier has an outer timeout to prevent one slow service from blocking the entire request.
- Repeated CLIP analysis of the same image is served from cache (clip_cache table) to reduce load and latency. Repeated Places API lookups for the same coordinates are also cached (places_cache table).

Security

- The system authenticates users with bcrypt-hashed credentials and authorizes all protected actions using signed JWT tokens.
- The system accepts only JPEG, PNG, and WebP image formats for analysis.
- The system rejects uploads that exceed 25 MB before processing begins (enforced at the Nginx layer).
- The system validates all structured request input through Pydantic schema validation and rejects malformed input before processing.

- The system restricts access to private user data (saved places, journals, profile, settings) to the authenticated owner.
- Administrative functions are accessible only to users with an administrator role.
- Uploaded images are stored under server-generated UUIDs, not client-supplied filenames, to prevent path traversal attacks.

Usability

- The system shall be usable without technical knowledge.
- A user shall be able to upload an image and start a location analysis without external help.
- The system shall present results (candidate locations, journal entries, gallery) in a clear and comparable format including a map view and ranked cards.
- A first-time user should be able to learn the main functions within 5 minutes.
- The interface shall remain usable on both desktop and mobile browsers.

Legal

- The system shall protect user privacy when handling uploaded images and derived location data.
- Location and image data shall be used only for user-requested features (image analysis, journal generation, gallery).
- External APIs shall be used in accordance with their applicable terms of service.
- Users are responsible for ensuring that uploaded images do not violate copyright or usage rights.

Performance

- Standard interactive requests (login, gallery browsing, profile, settings) shall normally complete within 2–3 seconds.
- Image location analysis, which calls multiple external AI and map APIs in sequence, shall normally complete within approximately 30 seconds. Repeat analyses of the same image benefit from caching and return faster.
- Journal generation runs as a background job so it does not block the user interface. The frontend polls for job completion.
- The system supports multiple concurrent users at demo deployment scale. Horizontal scaling is possible by running additional backend instances behind a load balancer.

3. System Architecture

3.1 Overview

Travel From Photo is a three-tier web application composed of a React single-page application (SPA) frontend, a FastAPI backend, and a PostgreSQL relational database. The three components are containerized and orchestrated using Docker Compose for consistent deployment across environments.

The system's core technical contribution is a multi-signal, tiered location inference pipeline in the backend. Rather than relying on a single model or API, the backend collects evidence from several independent sources and escalates through tiers only when the current tier's confidence is insufficient:

- **Tier 0 — EXIF GPS resolution:** If the uploaded image contains embedded GPS coordinates, the location is resolved directly using reverse geocoding and the Google Places API, bypassing all downstream tiers.
- **Tier 1 — Visual signal collection:** Google Vision API is queried in parallel for landmark detection, logo detection, label detection, OCR text, and web entity matching. Results are collected and normalized into candidate location signals. CLIP (a local vision-language model) provides additional visual similarity scoring.
- **Tier 2 — GPT main voter:** OpenAI Vision (GPT-4.1-mini) analyzes the image and proposes candidate locations based on all collected signals.
- **Tier 3 — GPT arbiter:** A second OpenAI call re-ranks the merged candidate set to produce a final confidence-ordered result.

Each tier has an individual timeout and falls back to best-available candidates if a service is slow or unavailable. The final response always includes multiple ranked candidates with a confidence verdict rather than a single forced answer.

In addition to location search, the backend provides a journal generation pipeline (CLIP scene classification + GPT-4.1-mini text generation running as a background job), a gallery and

saved-places system, tag-based live chat with WebSocket support, and an admin panel for user and moderation management.

3.2 Technologies and Version Numbers

Frontend

Technology	Version	Purpose
React	19.2	UI component framework
TypeScript	5.9	Static type checking
Vite	7.3	Build tool and dev server
React Router	7.15	Client-side routing (SPA navigation)
Leaflet / React-Leaflet	1.9 / 5.0	Interactive map rendering
Tailwind CSS	4.2	Utility-first styling
lucide-react	1.9	Icon library
exifr	7.1	Client-side EXIF metadata extraction

Backend

Technology	Version	Purpose
Python	3.11	Primary backend language
FastAPI	latest	HTTP API framework (async-capable)
Uvicorn	latest	ASGI server

SQLAlchemy	2.x	ORM and query builder
Alembic	latest	Database migration management
psycopg (v3)	latest	PostgreSQL driver
Pydantic	v2	Request/response schema validation
python-jose	latest	JWT token signing and verification
passlib / bcrypt	latest / 3.2.2	Password hashing
PyTorch	latest	CLIP model inference runtime
Hugging Face Transformers	latest	CLIP model loading (ViT-B/32)
OpenCV (headless)	latest	Image preprocessing
Pillow	latest	Image format handling
openai SDK	latest	OpenAI API client

Database and Infrastructure

Technology	Version	Purpose
PostgreSQL	14+	Primary relational database
Docker / Docker Compose	24.x / v2	Container orchestration
Nginx	1.27	Static file serving + API reverse proxy + WebSocket upgrade

External Services

Service	Usage
OpenAI API (GPT-4.1-mini)	Image location reasoning (Tier 2 and Tier 3 of search pipeline), journal text generation

Google Vision API	Landmark detection, label detection, logo detection, OCR, web entity detection (Tier 1)
Google Maps API	Reverse geocoding, Places enrichment (used in Tier 0 GPS resolution and candidate normalization)

3.3 Third-Party Components

OpenAI API

Used in two distinct pipelines. In the Search pipeline, GPT-4.1-mini acts as both the main location voter (Tier 2) and the final arbiter that re-ranks candidates (Tier 3). In the Journal pipeline, it generates a narrative travel note per photo entry based on CLIP scene labels and GPS-derived location context. The vision model is configurable via environment variable so it can be upgraded without code changes.

Google Vision API

Provides five parallel signal types in Tier 1 of the Search pipeline: landmark detection identifies recognizable structures; logo detection finds branded locations; label detection gives general scene labels; OCR extracts readable text (e.g., shop signs, street signs); and web detection finds visually matching pages. All five are queried concurrently to minimize latency.

Google Maps Platform (Geocoding + Places)

Used to normalize raw coordinates and signal strings into structured place data. The reverse geocoding API converts GPS coordinates to human-readable addresses. The Places API enriches candidates with official business names, categories, and coordinates.

CLIP API

A vision-language model that runs locally on the server. Used in two contexts: in the Search pipeline as an additional visual similarity scorer in Tier 1, and in the Journal pipeline to classify each photo into subject, atmosphere, and activity categories. The model is loaded once at server startup using Python's @lru_cache decorator and reused across all requests (Singleton pattern).

Docker and Docker Compose

Used to package all three services (frontend, backend, database) into isolated containers with consistent environments. Docker Compose defines the service topology, health checks, and shared volumes for uploaded files and database persistence.

Alembic

SQLAlchemy's migration tool, used to manage all database schema changes. The backend automatically runs `alembic upgrade head` on startup via `start.sh`, so schema changes deploy without manual intervention.

3.4 Software Design Patterns

Layered Architecture (Backend)

The backend is organized into four layers with dependencies flowing in one direction only (Router → Service → Repository → Database):

- **Router layer (app/routers/):** Receives HTTP requests, validates input via Pydantic schemas, and delegates to services. Does not contain business logic.
- **Service layer (app/services/):** Contains all business logic, organized into `search/`, `journal/`, and `shared/` sub-packages. Each external API call is isolated in its own service module.
- **Repository layer (app/repositories/):** Encapsulates all database read and write operations for journals and image metadata, keeping SQL out of the router and service layers.
- **Model/Schema layer (app/models/, app/schemas/):** Defines SQLAlchemy ORM models and Pydantic request/response contracts.

Single-Page Application (Frontend)

The frontend uses React Router for client-side navigation. All page transitions happen without full-page reloads, keeping the application state (search session, auth context) intact across navigation.

Singleton — Cached Model Loading

The CLIP model loader in `shared/clip_service.py` is wrapped with `@lru_cache(maxsize=1)`. This ensures the large model (approximately 330 MB) is instantiated only once per server process and reused across all requests.

Background Job Pattern

Journal generation is a long-running operation involving CLIP inference for multiple images followed by multiple OpenAI API calls. It runs in a background thread so the HTTP response returns immediately with a job ID. The frontend polls GET /api/journals/jobs/{job_id} until the job status becomes done or failed.

Graceful Degradation

Each tier of the Search pipeline has an outer timeout. If a tier times out or a service returns an error, the pipeline falls back to the best candidates collected so far and continues to the next tier. This ensures the system always returns a usable response even when individual external services are unavailable.

3.5 Technology Stack Rationale

FastAPI was chosen over alternatives like Django or Flask because it is natively async, includes automatic OpenAPI documentation generation, and uses Pydantic for type-safe request validation out of the box. These properties reduce boilerplate and make it straightforward to integrate async external API calls in the Search pipeline.

React with TypeScript was chosen for the frontend to enforce type safety across API response types and component props, which reduces runtime errors when integrating with the backend. Vite provides significantly faster hot module replacement than Create React App, which improves development speed.

PostgreSQL was selected as the database because the schema is relational (users → journals → journal entries, users → saved places, etc.) and benefits from foreign key constraints and transactional guarantees. JSONB columns are used where flexible schema is needed (CLIP labels, raw EXIF metadata, journal skip logs) without sacrificing indexability.

CLIP (local inference) was used instead of sending every image to an external API because it is deterministic, has zero per-call cost, and can be cached aggressively. Running it locally also removes a network round-trip for a frequently called step in both the Search and Journal pipelines.

2. Data Design

2.1 Database Overview

The system uses a PostgreSQL relational database managed through SQLAlchemy ORM and Alembic migrations. The schema consists of 12 tables covering user authentication, image analysis, gallery management, journal generation, social features, and pipeline caching.

2.2 Entity Descriptions

users

Stores registered user accounts. Each user has a unique email and a separate unique user_id (display handle). The role field distinguishes regular travelers from administrators. is_active enables account suspension without deletion.

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Internal surrogate key
user_id	VARCHAR(50)	UNIQUE, NOT NULL	User-chosen display handle

email	VARCHAR(255)	UNIQUE, NOT NULL	Login email
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash
first_name	VARCHAR(100)	NOT NULL	
last_name	VARCHAR(100)	NOT NULL	
role	VARCHAR(20)	NOT NULL, default 'traveler'	'traveler' or 'admin'
is_active	BOOLEAN	NOT NULL, default true	Account enabled/disabled
created_at	TIMESTAMPTZ	NOT NULL, server default	
updated_at	TIMESTAMPTZ	NOT NULL, auto-updated	

otps

Stores one-time passwords for email verification during registration. Each record expires after a set time window.

Column	Type	Constraints	Description
id	INTEGER	PK	
email	VARCHAR(255)	NOT NULL	Target email
code	VARCHAR(10)	NOT NULL	6-digit OTP
expires_at	TIMESTAMPTZ	NOT NULL	Expiration time

created_at	TIMESTAMPTZ	NOT NULL	

---

image_metadata

Records EXIF and file metadata for every image uploaded to the Search pipeline. This table serves as the source of truth linking an uploaded photo to its derived analysis data. The tags JSON column stores lounge tag keys derived from the search analysis result (e.g., ["historical", "urban"]), which are used to recommend chat lounges.

Column	Type	Constraints	Description
id	BIGINT	PK, auto-increment	
user_id	INTEGER	FK → users(id), nullable	Owning user (nullable for anonymous uploads)
file_name	VARCHAR(255)	NOT NULL	Original filename
absolute_path	VARCHAR(500)	nullable	Server filesystem path
file_size_bytes	BIGINT	NOT NULL	
image_format	VARCHAR(50)	nullable	e.g., JPEG, PNG
image_mode	VARCHAR(50)	nullable	e.g., RGB
width / height	INTEGER	nullable	Pixel dimensions
captured_at	VARCHAR(100)	nullable	EXIF DateTimeOriginal
camera_make / camera_model	VARCHAR	nullable	EXIF camera info
lens_model	VARCHAR(150)	nullable	

latitude / longitude	NUMERIC(9,6)	nullable	EXIF GPS coordinates
has_gps	BOOLEAN	NOT NULL	Whether GPS was extracted
metadata_case	VARCHAR(20)	NOT NULL	'gps_present' or 'gps_missing'
raw_metadata	JSON	nullable	Full EXIF dump
tags	JSON	nullable	Derived lounge tag keys
created_at	TIMESTAMPTZ	NOT NULL	

saved_places

Represents locations saved by a user from search results. Each saved place belongs to a named collection (a user-defined grouping). The image_url stores the HTTP-accessible path to the uploaded gallery image. The privacy column reflects the user's default privacy setting at the time of saving.

Column	Type	Constraints	Description
id	BIGINT	PK, auto-increment	
user_id	INTEGER	FK → users(id), NOT NULL	Owner
collection_name	VARCHAR(120)	NOT NULL, default 'My Gallery'	User-defined grouping
place_name	VARCHAR(255)	NOT NULL	Display name
formatted_address	VARCHAR(500)	nullable	Full address string
country	VARCHAR(100)	nullable	

city	VARCHAR(100)	nullable	
latitude / longitude	NUMERIC(9,6)	nullable	
image_filename	VARCHAR(255)	nullable	Stored filename
image_url	VARCHAR(500)	nullable	Served HTTP path
image_metadata_id	BIGINT	FK → image_metadata(id), nullable	Link back to analysis
privacy	VARCHAR(20)	NOT NULL, default 'private'	'private', 'unlisted', 'public'
created_at	TIMESTAMPTZ	NOT NULL	

journals

A journal represents a single generation job that processes one or more images into a set of travel log entries. The status column tracks the lifecycle of the background job. skipped stores a JSON array of images that could not be processed during generation, along with standardized reason codes.

Column	Type	Constraints	Description
id	BIGINT	PK, auto-increment	
user_id	INTEGER	FK → users(id), NOT NULL	Owner
title	VARCHAR(255)	nullable	User-set title
summary	TEXT	nullable	Auto-generated summary (future use)
visibility	VARCHAR(20)	NOT NULL, default 'private'	'private' or 'public'

status	VARCHAR(20)	NOT NULL, indexed	'pending', 'processing', 'done', 'partial_success', 'failed'
error_reason	TEXT	nullable	Set when status is 'failed'
skipped	JSONB	nullable	Array of {image_id, reason}
created_at	TIMESTAMPTZ	NOT NULL	
updated_at	TIMESTAMPTZ	NOT NULL, auto-updated	

journal_entries

One row per photo within a journal. Stores both the structured analytical output (CLIP labels, GPT categorical fields) and the final narrative journal text. The unique constraint on (journal_id, image_id) prevents duplicate entries during background job retries.

Column	Type	Constraints	Description
id	BIGINT	PK, auto-increment	
journal_id	BIGINT	FK → journals(id), NOT NULL	Parent journal
image_id	BIGINT	FK → image_metadata(id), NOT NULL	Source image
entry_order	INTEGER	NOT NULL	Position in the journal (0-based)

place_name	VARCHAR(255)	nullable	Resolved place name
country / city	VARCHAR	nullable	
address	VARCHAR(500)	nullable	
latitude / longitude	NUMERIC(9,6)	nullable	
captured_at	TIMESTAMPTZ	nullable	From EXIF
clip_subject	JSONB	nullable	CLIP subject label list
clip_atmosphere	JSONB	nullable	CLIP atmosphere label list
clip_activity	JSONB	nullable	CLIP activity label list
gpt_shooting_style	VARCHAR(50)	nullable	GPT categorical
gpt_subject_focus	VARCHAR(50)	nullable	GPT categorical
gpt_time_of_day	VARCHAR(50)	nullable	GPT categorical
gpt_atmosphere	VARCHAR(50)	nullable	GPT categorical
gpt_weather_light	VARCHAR(50)	nullable	GPT categorical
gpt_composition_habit	VARCHAR(50)	nullable	GPT categorical
gpt_color_mood	VARCHAR(50)	nullable	GPT categorical
gpt_cultural_layer	VARCHAR(50)	nullable	GPT categorical

gpt_detail_note	TEXT	nullable	GPT free-text observation
journal_text	TEXT	nullable	Final narrative paragraph
generated_by	VARCHAR(20)	NOT NULL	'clip_gpt', 'manual', or 'cache'
model_version	VARCHAR(100)	nullable	OpenAI model used
vocab_version	VARCHAR(20)	nullable	CLIP vocab version
generated_at	TIMESTAMPTZ	NOT NULL	

Unique constraint: (journal_id, image_id)

user_settings

One-to-one extension of the users table storing configurable preferences. The default_privacy value is automatically applied when a user saves a new gallery entry.

Column	Type	Constraints	Description
id	INTEGER	PK	
user_id	INTEGER	FK → users(id), UNIQUE, NOT NULL	One record per user
display_name	VARCHAR(120)	NOT NULL	Shown in nav and chat

default_privacy	VARCHAR(20)	NOT NULL, default 'private'	Applied to new gallery saves
theme	VARCHAR(20)	NOT NULL, default 'system'	'system', 'light', 'dark'
email_notifications	BOOLEAN	NOT NULL, default true	
bio	TEXT	nullable	
created_at / updated_at	TIMESTAMPTZ	NOT NULL	

chat_rooms

Represents one of 13 permanent tag-based chat lounges. Rooms are seeded at deployment and remain constant. Each room has a tag_key that matches the tags produced by the Search pipeline, enabling automatic lounge recommendations after a search.

Column	Type	Constraints	Description
id	INTEGER	PK	
tag_key	VARCHAR(80)	UNIQUE, NOT NULL	e.g., 'historical', 'food', 'urban'
display_name	VARCHAR(120)	NOT NULL	Human-readable name
emoji	VARCHAR(16)	NOT NULL	Room icon
description	TEXT	NOT NULL	Short description
category	VARCHAR(40)	NOT NULL	'nature', 'urban', 'culture', 'experience'
created_at	TIMESTAMPTZ	NOT NULL	

chat_messages

Stores all messages sent in chat lounges. image_url optionally links to a gallery photo the sender chose to share. image_id optionally links to the corresponding image_metadata record.

Column	Type	Constraints	Description
id	INTEGER	PK	
room_id	INTEGER	FK → chat_rooms(id), NOT NULL	Target lounge
sender_user_id	INTEGER	FK → users(id), NOT NULL	Sender
message_text	TEXT	NOT NULL	Message content (max 1000 chars)
image_id	BIGINT	FK → image_metadata(id), nullable	Attached image metadata
image_url	TEXT	nullable	Served URL of attached gallery photo
created_at	TIMESTAMPTZ	NOT NULL	
read_at	TIMESTAMPTZ	nullable	

moderation_items

Used for both user-submitted bug reports and admin-created moderation cases. All records flow into the same admin queue regardless of origin. The item_type field distinguishes categories (e.g., 'bug', 'chat', 'search result').

Column	Type	Constraints	Description
id	INTEGER	PK	
item_type	VARCHAR(80)	NOT NULL	Category of the report

title	VARCHAR(200)	NOT NULL	Brief summary
reporter_name	VARCHAR(120)	NOT NULL	Display name of reporter
reason	TEXT	NOT NULL	Detailed description
status	VARCHAR(20)	NOT NULL, default 'open'	'open' or 'resolved'
created_at	TIMESTAMPTZ	NOT NULL	
resolved_at	TIMESTAMPTZ	nullable	Set when status changes to 'resolved'

clip_cache

Caches CLIP inference results keyed by (image_id, vocab_version). Since CLIP is deterministic for a given image and vocabulary, results can be reused indefinitely without re-running inference. Bumping the vocab_version automatically invalidates all cached results for that version.

Column	Type	Constraints	Description
image_id	BIGINT	PK, FK → image_metadata(id)	
vocab_version	VARCHAR(20)	PK	CLIP vocabulary version string
clip_subject	JSONB	nullable	Cached subject label list
clip_atmosphere	JSONB	nullable	Cached atmosphere label list
clip_activity	JSONB	nullable	Cached activity label list
created_at	TIMESTAMPTZ	NOT NULL	

Composite primary key: (image_id, vocab_version)

places_cache

Caches Google Places API results keyed by rounded GPS coordinates (4 decimal places, approximately 11 meters of precision). Different photos taken near the same location reuse one Places API response, reducing API call volume.

Column	Type	Constraints	Description
rounded_lat	NUMERIC(9,4)	PK	GPS latitude rounded to 4 decimals
rounded_lng	NUMERIC(9,4)	PK	GPS longitude rounded to 4 decimals
payload	JSONB	nullable	Raw Places API response
created_at	TIMESTAMPTZ	NOT NULL	

Composite primary key: (rounded_lat, rounded_lng)

2.3 Key Relationships

users (1) ——— (N) image_metadata
users (1) ——— (N) saved_places
users (1) ——— (N) journals
users (1) ——— (1) user_settings
users (1) ——— (N) chat_messages

journals (1) ——— (N) journal_entries
journal_entries (N) ——— (1) image_metadata

saved_places (N) ——— (1) image_metadata

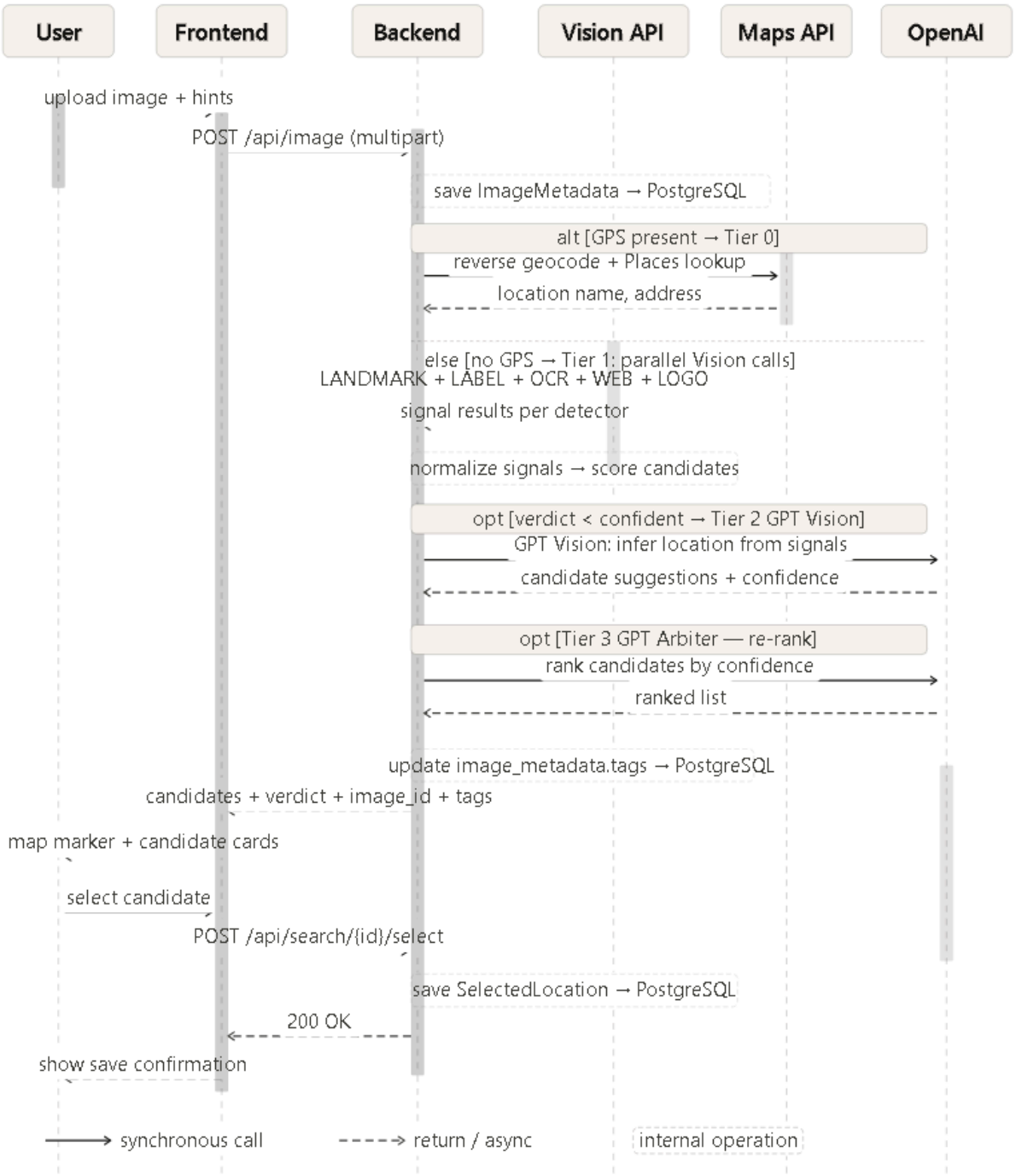
chat_rooms (1) ——— (N) chat_messages

image_metadata (1) ——— (1) clip_cache [per vocab version]

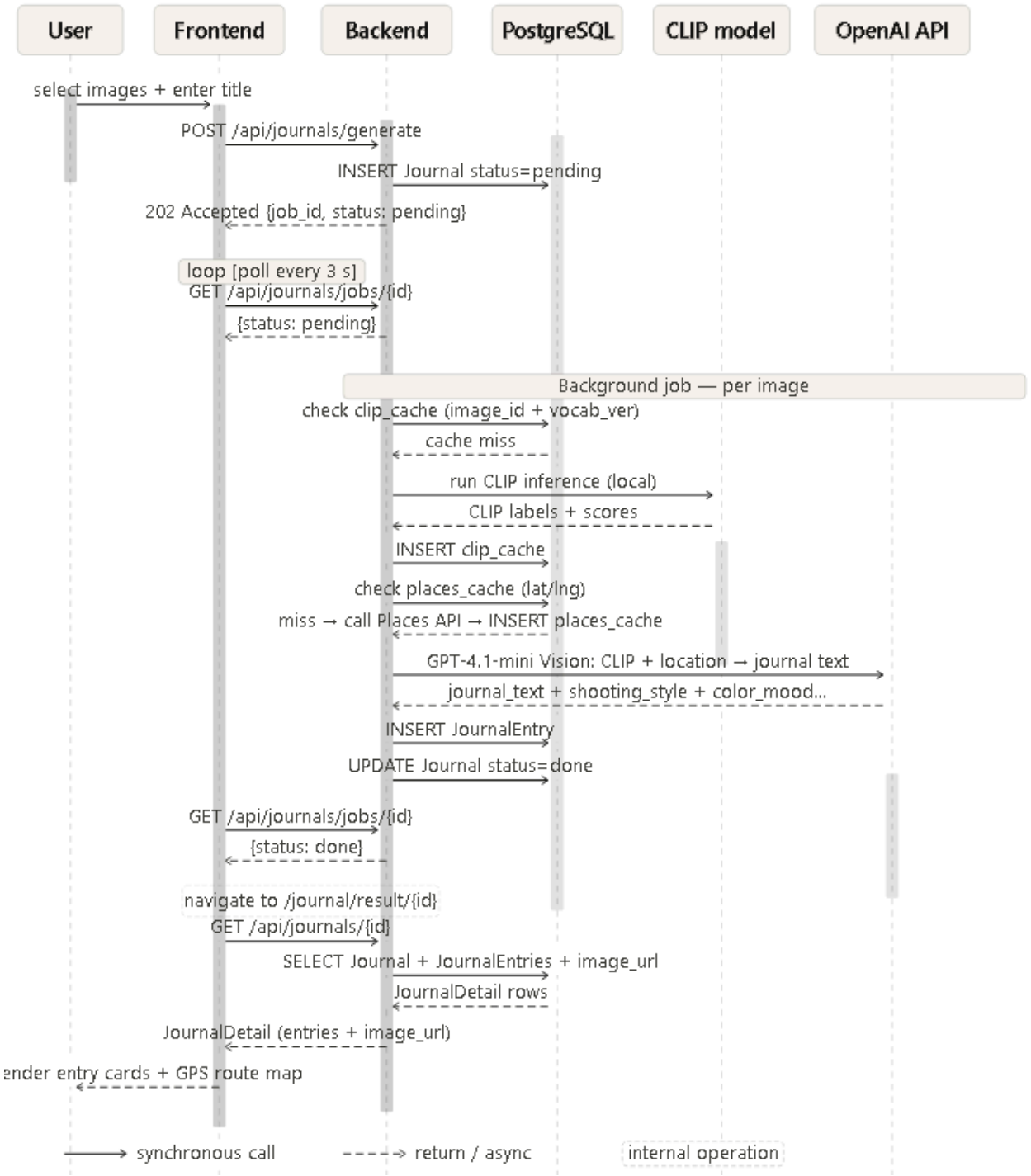
places_cache ——— [keyed by lat/lng, not FK]

3. UML Sequence Diagrams

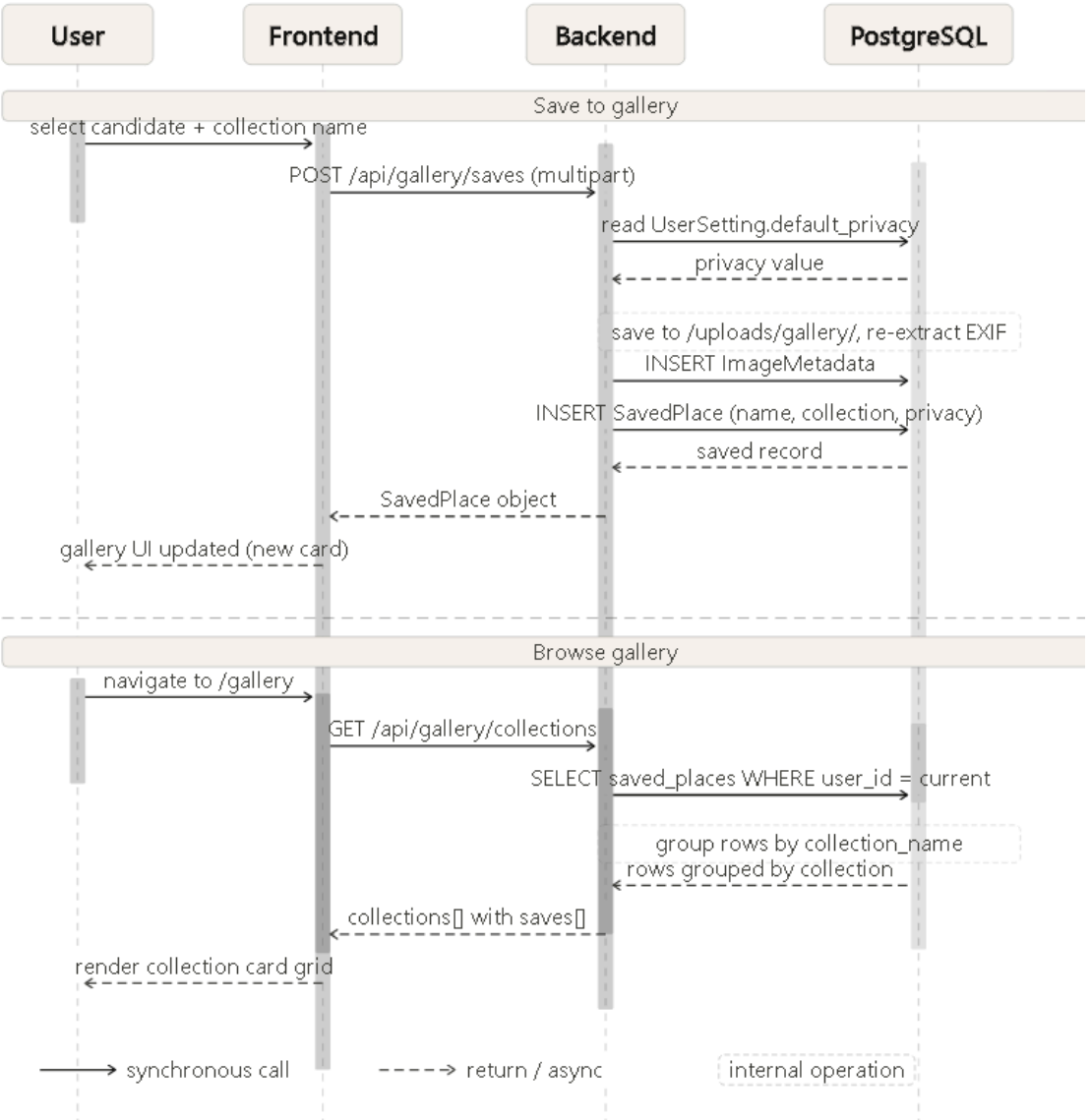
Search Flow



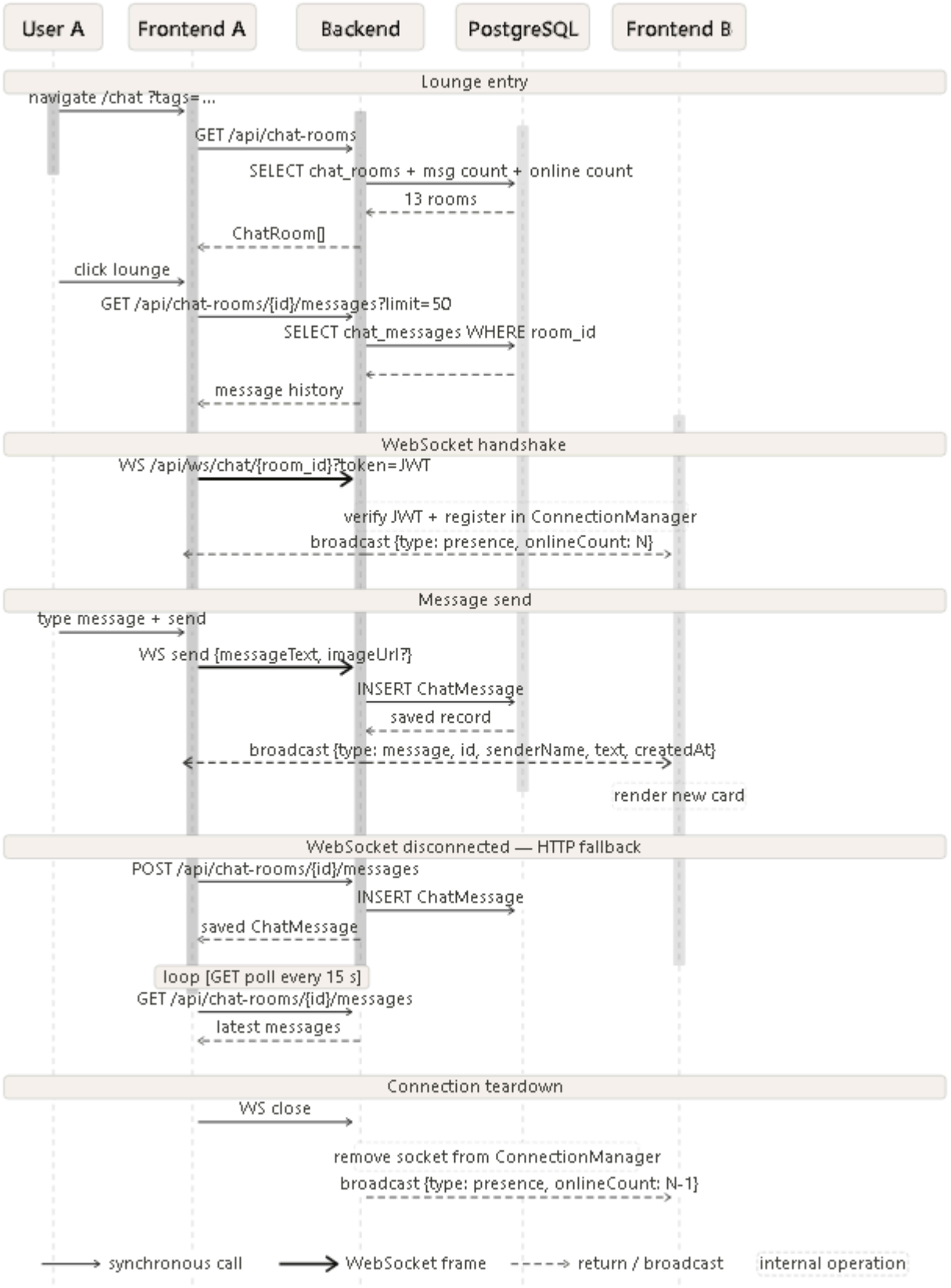
Journal Flow


User
Frontend
Backend
PostgreSQL
CLIP model
OpenAI API
select images + enter title
POST /api/journals/generate
INSERT Journal status=pending
202 Accepted {job_id, status: pending}
loop [poll every 3 s]
GET /api/journals/jobs/{id}
{status: pending}
Background job — per image
check clip_cache (image_id + vocab_ver)
cache miss
run CLIP inference (local)
CLIP labels + scores
INSERT clip_cache
check places_cache (lat/lng)
miss → call Places API → INSERT places_cache
GPT-4.1-mini Vision: CLIP + location → journal text
journal_text + shooting_style + color_mood...
INSERT JournalEntry
UPDATE Journal status=done
GET /api/journals/jobs/{id}
{status: done}
navigate to /journal/result/{id}
GET /api/journals/{id}
SELECT Journal + JournalEntries + image_url
JournalDetail rows
JournalDetail (entries + image_url)
ender entry cards + GPS route map
synchronous call
return / async
internal operation

Gallery



Live Chat Flow


User A
Frontend A
Backend
PostgreSQL
Frontend B
Lounge entry
navigate /chat ?tags=...
GET /api/chat-rooms
SELECT chat_rooms + msg count + online count
13 rooms
ChatRoom[]
click lounge
GET /api/chat-rooms/{id}/messages?limit=50
SELECT chat_messages WHERE room_id
message history
WebSocket handshake
WS /api/ws/chat/{room_id}?token=JWT
verify JWT + register in ConnectionManager
broadcast {type: presence, onlineCount: N}
Message send
type message + send
WS send {messageText, imageUrl?}
INSERT ChatMessage
saved record
broadcast {type: message, id, senderName, text, createdAt}
render new card
WebSocket disconnected — HTTP fallback
POST /api/chat-rooms/{id}/messages
INSERT ChatMessage
saved ChatMessage
loop [GET poll every 15 s]
GET /api/chat-rooms/{id}/messages
latest messages
Connection teardown
WS close
remove socket from ConnectionManager
broadcast {type: presence, onlineCount: N-1}
synchronous call
WebSocket frame
return / broadcast
internal operation

4. API Design

All endpoints are prefixed with /api. Unless stated otherwise, all endpoints require a valid JWT token in the Authorization: Bearer <token> header. On success, the HTTP status is 200 unless noted.

4.1 Authentication

Register a new account

- **Method:** POST
- **URL:** /api/auth/register
- **Auth required:** No
- **Request body:**

{ "first_name": "string (1–100 chars)",

"last_name": "string (1–100 chars)",

"user_id": "string (3–50 chars, unique handle)",

"email": "string (valid email)",

"password": "string (8–72 chars, must contain letter and digit)"}

- **Response (200):**

{ "message": "OTP sent to your email. Please verify to complete registration." }

- **Notes:** Sends a 6-digit OTP to the provided email. Account is not activated until the OTP is verified.

Verify OTP to activate account

- **Method:** POST
- **URL:** /api/auth/verify-otp
- **Auth required:** No
- **Request body:**

{ "email": "string", "code": "string (6 digits)" }

- **Response (200):**

{ "message": "Account verified successfully." }

- **Notes:** OTP expires after a short window. Re-register to receive a new code.

Login

- **Method:** POST
- **URL:** /api/auth/login
- **Auth required:** No
- **Request body:**

{ "email": "string", "password": "string" }

- **Response (200):**

{ "access_token": "string (JWT)",

"token_type": "bearer",

"user_id": "string",

"role": "traveler | admin",

"first_name": "string",

"last_name": "string",

"email": "string"}

- **Notes:** The access_token must be included in the Authorization header for all protected endpoints. Token expiry is configurable via JWT_EXPIRE_MINUTES.

Get current user (minimal)

- **Method:** GET
- **URL:** /api/auth/me
- **Auth required:** Yes
- **Response (200):**

{ "user_id": "string",

"role": "string",

"first_name": "string",

"last_name": "string",

"email": "string"}

4.2 Profile and Settings

Get full profile

- **Method:** GET
- **URL:** /api/users/me or /api/profile
- **Auth required:** Yes
- **Response (200):**

{ "firstName": "string",

"lastName": "string",

"userId": "string",

"email": "string",

"bio": "string | null",

"displayName": "string",

"defaultPrivacy": "private | unlisted | public",

"theme": "system | light | dark"}

Update profile

- **Method:** PUT
- **URL:** /api/profile
- **Auth required:** Yes
- **Request body:**

{ "firstName": "string (1–100 chars)",

"lastName": "string (1–100 chars)",

"email": "string (valid email)",

"bio": "string (max 500 chars) | null"}

- **Response (200):** Same as Get full profile response.
- **Notes:** Returns 409 if the new email is already in use by another account.

Get settings

- **Method:** GET
- **URL:** /api/settings
- **Auth required:** Yes
- **Response (200):**

{ "displayName": "string",

"defaultPrivacy": "private | unlisted | public", "theme": "system | light | dark",

"emailNotifications": true}

Update settings

- **Method:** PATCH
- **URL:** /api/settings
- **Auth required:** Yes
- **Request body:**

{

"displayName": "string (1–120 chars)",

"defaultPrivacy": "private | unlisted | public",

"theme": "system | light | dark",

"emailNotifications": true

}

- **Response (200):** Same structure as Get settings.

- **Notes:** defaultPrivacy is automatically applied to new gallery saves after this is set.

4.3 Search (Image Location Analysis)

Analyze an uploaded image

- **Method:** POST
- **URL:** /api/image
- **Auth required:** No (but user_id is recorded if authenticated)
- **Request:** multipart/form-data

Field	Type	Required	Description
file	file	Yes	Image file (JPEG, PNG, WebP, max 25 MB)
country_hint	string	No	Country name hint to improve accuracy
city_hint	string	No	City name hint
user_hint	string	No	Free-text hint from the user
force_openai_retry	boolean	No	Forces re-analysis using GPT even if a confident result was found earlier

- **Response (200):**

{"image_id":42,"metadata_case":"gps_present | gps_missing","has_gps":false,"verdict":"confident | likely | uncertain | low_signal","candidates":[{"rank":1,"place_name":"Eiffel Tower","formatted_address":"Champ de Mars, 5 Av. Anatole France, Paris","country":"France","city":"Paris","latitude":48.8584,"longitude":2.2945,"aggregated_score":0.91,"contributing_sources":["landmark","gpt_main","gpt_arbiter"]}],"signal_summary":{},"tags":["historical","urban"]}

- **Notes:** This is the most latency-intensive endpoint (~30 seconds). The pipeline runs up to 4 tiers (EXIF GPS → Tier 1 Vision signals → GPT main voter → GPT arbiter). Earlier tiers short-circuit if a high-confidence result is found. tags contains derived lounge tag keys used to recommend chat rooms.

4.4 Gallery

List all saved places grouped by collection

- **Method:** GET
- **URL:** /api/gallery/collections
- **Auth required:** Yes
- **Response (200):**

{"collections":[{"name":"Japan Trip","saves":[{"id":1,"collection_name":"Japan Trip","place_name":"Fushimi Inari Taisha","formatted_address":"68 Fukakusa Yabunouchicho, Fushimi Ward, Kyoto","country":"Japan","city":"Kyoto","latitude":34.9671,"longitude":135.7727,"image_url":"/uploads/gallery/abc123.jpg","image_metadata_id":42,"has_gps":true,"privacy":"private","created_at":"2026-05-01T12:00:00Z"}]}]}

Save a location to gallery

- **Method:** POST
- **URL:** /api/gallery/saves
- **Auth required:** Yes
- **Request:** multipart/form-data

Field	Type	Required	Description
image	file	Yes	Photo to save (JPEG, PNG, WebP)
place_name	string	Yes	Display name for the saved place

collection_name	string	No	Default: "My Gallery"
formatted_address	string	No	
country	string	No	
city	string	No	
latitude	float	No	
longitude	float	No	

- **Response (200):** Single SavedPlace object (same structure as items in the collections response).
- **Notes:** The image is stored on the server under a UUID filename. privacy is automatically set from the user's defaultPrivacy setting.

Update a saved place

- **Method:** PATCH
- **URL:** /api/gallery/saves/{save_id}
- **Auth required:** Yes
- **Request body:**

{ "place_name": "string | null","collection_name": "string | null","latitude": "float | null", "longitude": "float | null", "formatted_address": "string | null"}

- **Response (200):** Updated SavedPlace object.

Delete a saved place

- **Method:** DELETE
- **URL:** /api/gallery/saves/{save_id}

- **Auth required:** Yes
- **Response (200):**

{ "deleted_id": 1 }

Rename a collection

- **Method:** POST
- **URL:** /api/gallery/collections/rename
- **Auth required:** Yes
- **Request body:**

{ "old_name": "Japan Trip", "new_name": "Japan 2025" }

- **Response (200):**

{ "renamed_count": 5, "new_name": "Japan 2025" }

Delete an entire collection

- **Method:** DELETE
- **URL:** /api/gallery/collections/{collection_name}
- **Auth required:** Yes
- **Response (200):**

{ "deleted_count": 5, "collection_name": "Japan Trip" }

4.5 Journal

Start a journal generation job

- **Method:** POST
- **URL:** /api/journals/generate
- **Auth required:** Yes
- **Status code on success:** 202 Accepted
- **Request body:**

{ "image_ids": [1, 2, 3], "title": "string | null" }

- **Response (202):**

{ "job_id": 7, "status": "pending" }

- **Notes:** Generation runs as a background task. Returns immediately with a job_id. Poll /api/journals/jobs/{job_id} to track progress. Only one active job is allowed per user at a time (returns 409 if another is running).

Poll job status

- **Method:** GET
- **URL:** /api/journals/jobs/{job_id}
- **Auth required:** Yes
- **Response (200):**

{ "job_id": 7, "status": "pending | processing | done | partial_success | failed", "journal_id": 7, "entries_created": 3, "skipped": [{ "image_id": 5, "reason": "PLACES_NO_RESULT" }], "error": "string | null"}

- **Notes:** journal_id is populated when status is done or partial_success. Possible skip reasons: NO_METADATA, PLACES_API_TIMEOUT, PLACES_NO_RESULT, GPT_GENERATION_FAILED.

List all journals

- **Method:** GET
- **URL:** /api/journals
- **Auth required:** Yes
- **Response (200):**

[{ "id": 7,"title": "Kyoto Trip", "status": "done", "primary_city": "Kyoto","primary_country": "Japan", "entry_count": 4,"earliest_captured_at": "2025-03-12T09:30:00Z", "created_at": "2026-05-01T12:00:00Z","cover_image_url": "/uploads/gallery/abc123.jpg"}]

***Get journal detail**

- **Method:** GET
- **URL:** /api/journals/{journal_id}

- **Auth required:** Yes
- **Response (200):**

{ "id": 7, "title": "Kyoto Trip", "summary": null,"status": "done". "visibility": "private", "entries": [

{ "id": 21, "image_id": 3, "image_url": "/uploads/gallery/abc123.jpg", "entry_order": 0,"place_name": "Fushimi Inari", "country": "Japan","city": "Kyoto","latitude": 34.9671,"longitude": 135.7727,"captured_at": "2025-03-12T09:30:00Z","clip_subject": ["temple", "shrine"],"clip_atmosphere": ["serene", "spiritual"],"clip_activity": ["sightseeing"],"gpt_time_of_day": "morning","gpt_color_mood": "warm","journal_text": "Early morning at Fushimi Inari...", "generated_by": "clip_gpt", "generated_at": "2026-05-01T12:05:00Z" }]}

- **Edit journal title or entry text**
- **Method:** PATCH
- **URL:** /api/journals/{journal_id}
- **Auth required:** Yes
- **Request body:**

{ "title": "string | null",

"entries": [{ "id": 21, "journal_text": "Updated text..." }]}

- **Response (200):** Same as Get journal detail.

Delete a journal

- **Method:** DELETE
- **URL:** /api/journals/{journal_id}
- **Auth required:** Yes
- **Response:** 204 No Content

Get travel statistics

- **Method:** GET
- **URL:** /api/journals/stats
- **Auth required:** Yes

- **Response (200):**

{ "photo_count": 42,

"country_count": 5,

"city_count": 12,

"countries": ["Japan", "France"],

"cities": ["Kyoto", "Paris"],

"total_distance_km": 18500.0,

"subject_distribution": { "temple": 8, "street": 5 },

"atmosphere_distribution": { "serene": 10 },

"activity_distribution": { "sightseeing": 15 },

"cultural_layer_distribution": { "east_asian": 12 },

"color_mood_distribution": { "warm": 9 },

"composition_distribution": { "wide_angle": 7 },

"time_of_day_distribution": { "morning": 14, "afternoon": 10 }}

***Get AI-generated destination recommendations**

- **Method:** GET
- **URL:** /api/journals/recommendations
- **Auth required:** Yes
- **Response (200):**

{ "recommendations": [{ "name": "Nara", "country": "Japan", "reason": "Based on your preference for temples and serene atmospheres..." }], "low_data": false,

"model_version": "gpt-4.1-mini"}

- **Notes:** GPT analyzes the user's journal history and travel patterns to suggest next destinations. low_data is true when the user has fewer than 3 journal entries, in which case recommendations may be less precise.

4.6 Live Chat

List all 13 tag lounges

- **Method:** GET
- **URL:** /api/chat-rooms
- **Auth required:** Yes
- **Response (200):**

{"items":[{"id":1,"tagKey":"historical","displayName":"Historical & Heritage","emoji":"🏛️","description":"Historic places, palaces, temples, castles...","category":"urban","onlineCount":3,"messageCount":42}]}

***Get recommended lounges for search tags**

- **Method:** GET
- **URL:** /api/chat-rooms/recommendations
- **Auth required:** Yes
- **Query parameters:**

Param	Type	Description
tags	string	Comma-separated tag keys (e.g., historical,urban)
imageId	integer	If provided, uses that image's stored tags instead

- **Response (200):** Same structure as List all lounges, filtered to matching rooms.

Get messages in a lounge

- **Method:** GET
- **URL:** /api/chat-rooms/{room_id}/messages

- **Auth required:** Yes
- **Query parameters:** limit (integer, 1–100, default 50)
- **Response (200):**

{"items":[{"id":"101","roomId":1,"roomTag":"historical","senderId":"jaemin001","senderName":"Jaemin Jeon","messageText":"Hello from Kyoto!","imageId":null,"imageUrl":null,"createdAt":"2026-05-30T10:00:00Z","readAt":null}]}

Send a message via REST (WebSocket fallback)

- **Method:** POST
- **URL:** /api/chat-rooms/{room_id}/messages
- **Auth required:** Yes
- **Request body:**

{ "messageText": "string (1–1000 chars)",

"imageId": "integer | null",

"imageUrl": "string | null"}

- **Response (200):** Single ChatMessage object (same structure as items above).
- **Notes:** imageUrl must be a URL from the sender's own gallery saves. The backend verifies ownership and returns 403 otherwise. Prefer the WebSocket connection for real-time messaging; use this endpoint as a fallback.

Normalize search analysis to lounge tags

- **Method:** POST
- **URL:** /api/chat-tags/normalize
- **Auth required:** Yes
- **Request body:**

{ "analysis": { ... } }

- **Response (200):**

{ "tags": ["historical", "urban"],

"lounges": [

{ "tag_key": "historical", "display_name": "Historical & Heritage", "emoji": "🏛️", ... }]}

- **Notes:** analysis is the raw JSON response from POST /api/image. The backend extracts keywords and maps them to the 13 standard tag keys.

WebSocket — Real-time chat

- **Method:** WebSocket (GET upgrade)
- **URL:** /api/ws/chat/{room_id}?token=<JWT>
- **Auth required:** Token passed as query parameter (WebSocket does not support Authorization header)
- **Incoming messages (client → server):**

{ "messageText": "string", "imageUrl": "string | null" }

- **Outgoing messages (server → client):**

Type	Payload
message	{ "type": "message", "message": { ChatMessage } }
presence	{ "type": "presence", "roomId": 1, "onlineCount": 4 }
error	{ "type": "error", "detail": "string" }

- **Notes:** The connection is closed with code 1008 if the token is invalid or the room does not exist. Presence events are broadcast when users connect or disconnect. Messages are broadcast to all active connections in the room.

4.7 Geocoding

Reverse geocode coordinates

- **Method:** GET
- **URL:** /api/geocode/reverse
- **Auth required:** No
- **Query parameters:** lat (float, required), lng (float, required)
- **Response (200):**

{ "place_name": "Fushimi Inari Taisha",

"formatted_address": "68 Fukakusa Yabunouchicho, Fushimi Ward, Kyoto",

"city": "Kyoto",

"country": "Japan",

"latitude": 34.9671,

"longitude": 135.7727}

- **Notes:** Returns 502 if the Google Maps Geocoding API call fails. If no result is found, all string fields are null but latitude and longitude are echoed back.

4.8 Bug Reports

Submit a bug report

- **Method:** POST
- **URL:** /api/reports
- **Auth required:** Yes
- **Status code on success:** 201 Created
- **Request body:**

{"title": "string (3–200 chars)",

"description": "string (5–2000 chars)"}

- **Response (201):**

{ "id": "15", "status": "open", "createdAt": "2026-06-01T10:00:00Z" }

- **Notes:** Reports are stored as ModerationItem records with item_type = "bug" and appear in the admin moderation queue. The reporter name is automatically taken from the authenticated user's display name.

4.9 Admin (Admin role required)

All endpoints in this section return 403 if the authenticated user does not have the admin role.

Get admin dashboard summary

- **Method:** GET
- **URL:** /api/admin/summary
- **Response (200):**


```
{
  "totalUsers": 120,
  "activeUsers": 115,
  "reviewUsers": 0,
  "disabledUsers": 5,
  "openModerationItems": 3,
  "totalChatMessages": 870
}
```


List users

- **Method:** GET
- **URL:** /api/admin/users
- **Query parameters:** q (string, optional — searches name, email, user_id)
- **Response (200):**

{"items":[{"id":"jaemin001","displayName":"Jaemin Jeon","email":"jaemin@example.com","role":"admin","status":"active","uploads":0,"journals":0,"lastActive":"2026-06-01T09:00:00Z"}]}

Update user role or status

- **Method:** PATCH
- **URL:** /api/admin/users/{user_id}
- **Request body:**

{ "role": "traveler | admin | null", "status": "active | disabled | null" }

-
- **Response (200):** Updated AdminUser object.
- **Notes:** An admin cannot disable their own account (returns 400).

List moderation items

- **Method:** GET
- **URL:** /api/admin/moderation
- **Response (200):**

{"items":[{"id":"15","type":"bug","title":"Search map does not load on mobile","reporter":"Mina Park","reason":"When I tap the map...","status":"open","createdAt":"2026-06-01T10:00:00Z"}]}

Create a moderation item (admin-initiated)

- **Method:** POST
- **URL:** /api/admin/moderation
- **Request body:**

{ "type": "string", "title": "string", "reporter": "string", "reason": "string" }

- **Response (200):** Created ModerationItem object.

Resolve a moderation item

- **Method:** PATCH

- **URL:** /api/admin/moderation/{item_id}
- **Request body:** None
- **Response (200):** Updated ModerationItem with status: "resolved" and createdAt timestamp.

4.10 Health Check

Backend health check

- **Method:** GET
- **URL:** /api/health
- **Auth required:** No
- **Response (200):**
- { "status": "ok" }

5. Deployment

For deployment, our team plans to use Amazon Web Services (AWS) EC2 with an Ubuntu server and Docker Compose. We chose this setup because our project already has three main parts: the React frontend, the FastAPI backend, and the PostgreSQL database. Since these parts are already organized with Docker, EC2 is a good option because we can run the whole project on one cloud server and access it through a public IP address.

The frontend will be built with React and Vite. In deployment, it will be served through an Nginx container. Nginx will show the frontend pages to users and also forward backend API requests to the FastAPI server. This way, users can access the website from one main address instead of using separate links for the frontend and backend.

The backend will run in a FastAPI container. It will handle login, image upload, OpenAI-based location search, gallery, journal, profile/settings, admin features, and live chat APIs. It will also support WebSocket connections for the tag-based Live Chat lounges.

For the database, we will use PostgreSQL in a Docker container. The database will store user accounts, profile/settings data, uploaded image metadata, search results, gallery data, journal data, chat rooms, chat messages, and moderation records. We will also use Alembic migrations so the database schema can be set up in the correct state when the project is deployed.

On the server, we will use a .env file to store private settings such as the OpenAI API key, JWT secret, database name, database username, and database password. These values should not be pushed to GitHub. Instead, our repository will include a .env.example file so other developers know what values are needed.

Our deployment process will be:

1. Merge the final project code into GitHub.
2. Create an AWS EC2 Ubuntu instance.
3. Open the needed ports, such as SSH port 22 and HTTP port 80.
4. Connect to the server using SSH.
5. Install Docker, Docker Compose, and Git.
6. Clone the GitHub repository on the server.
7. Create the .env file with the required values.
8. Start the project with docker compose up --build -d.
9. Run the seed script to create test accounts.
10. Open the public URL and test the main features.

For beta testing, we will use seed accounts for an admin user and a normal traveler user. After deployment, we will test the main flow of the application, including login, image upload, location search, gallery, journal, profile/settings, live chat lounges, and admin access. We will also check that chat messages are saved in PostgreSQL and still appear after refreshing the page or logging in again.

6. Code Conventions

We adhere to community-standard style guides per language, enforced by automated linters.

6.1 Python (backend, FastAPI)

Guide: PEP 8 — Style Guide for Python Code

- **Naming**: snake_case for modules, functions, variables; PascalCase for classes; SCREAMING_SNAKE_CASE for constants; leading _ for module-private members.
- **Formatting**: 4-space indentation, 100-char line limit, imports grouped as stdlib → 3rd-party → local.
- **Comments**: triple-quoted docstrings on public functions/classes; inline # comments explain *why*, not *what*.
- **Type hints** required on function signatures.
- **Checker**: ruff (lint + format).

6.2 TypeScript / React (frontend, Vite)

Guide: Airbnb JavaScript Style Guide + TypeScript ESLint recommended ruleset.

- **Naming**: camelCase for variables/functions, PascalCase for components, types, interfaces, and component files (GalleryPage.tsx); useCamelCase for hooks; boolean variables prefixed is/has/should.
- **Formatting**: 2-space indentation, 100-char line limit, single quotes (double for JSX attributes), trailing commas on multi-line literals.
- **Comments**: // for the *why*; JSDoc only for complex exported APIs.
- **Component rule**: one component per file, named export.
- **Checker**: eslint (config in eslint.config.js) + tsc --noEmit for type checking.

6.3 CSS

Guide: BEM-style class names (BEM Naming).

- **Naming**: block__element--modifier (e.g., journal-collection-card--active).
- **Formatting**: 2-space indentation, one declaration per line, lowercase hex colors.
- **Checker**: visual review via design system tokens defined in :root of index.css.

4. Schedule

Period	Task
Week 1	Project Ideation, Brainstorming, and Conceptualization
Week 2	Core Feature Definition, Tech Stack Selection, and Proposal Preparation
Week 3	Project Architecture Setup, API Exploration, and Scope Expansion Review
Week 4	UI / UX Mockup Design and API Feasibility Assessment
Week 5	UI Mockup Finalization, Search Logic Specification, and Initial Frontend Setup.
Week 6	Features Specification Drafting and Core Scope Freeze(Serach, Journal, Travelize, Gallery)
Week 7	Frontend Layout Completion and Module Pipeline Development(Phase 1)
Week 8	Multi-Track Feature Integration and Pipeline Optimization
Week 9	Authentication System Implementation and System Architecture Documentattion (UML & Data Design), Keep working on Pipeline
Week 10	Project Restructuring and Scope Realignment(Deprecation of Travelize Feature)
Week 11	Core Module Redesign(Search, Journal, Live Chat) and Iterative Implementation
Week 12	Gallery System Integration and Migration to Multi-Tier Hierarchical Pipeline Architecture
Week 13	Final Feature Polish and Algorithmic Logic Optimization for Search Accuracy

Week 14	Auxiliary Feature Completion(Live Chat, Profile, Settings) and End-to-End System Testing
Week 15	Comprehensiv Debugging, System Validation, and Production Deployment

5. Contributions

Member	Contribution
Jaemin Jeon	profile/settings, UI support, and Live chat implementation, integration testing.
Younghak Yoo	Search backend(Multi-API Hierarchical Scoring Pipeline), Journal Pipeline, Gallery system, and Overall Frontend
Both Members	Final testing, bug fixing, presentation preparation, and required documents revision, security enhancement